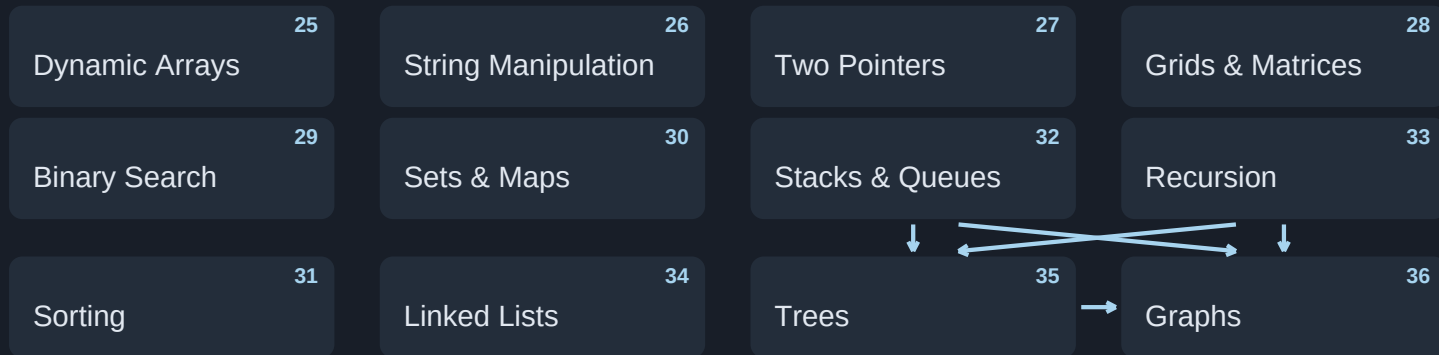


Tier 1 - Fundamental



Tier 2 - Common and important



Interview checklist

1 Understand the problem

- Read the statement twice
- Examples out loud
- Clarify edge cases, I/O and scale

2 Design the algorithm

- Sketch the naive solution
- Upper & lower bounds
- Look for triggers
- Use boosters

3 Explain the solution

- Examples
- Indented English
- Name & justify

4 Get buy-in

Should I start coding this up,
or would you like me to keep
looking for something better?

5 Code the solution

- Main logic
- Helper functions
- Stop if you get lost

6 Verify the solution

- Top-to-bottom pass
- Tricky expressions
- Trace a tiny input
- Check edge cases
- Verify time & space

Boosters - Help Developers Crack Algorithms

- B - Brute-force optimization
- H - Hunt for properties
- D - Decrease the difficulty
- C - Cycle through the catalog
- A - Articulate your blocker

Big O of ...

Complexity / typical growth

Examples

Approx. max n
conservative / generous

$O(1)$ - Constant (one)

Bounded work as n grows.

Array access
 $\min(n, 10)$

No n-based limit

$O(\log n)$ - Logarithmic ($\log n$)

Double n: add constant work.

Binary search
 $\log_2(n)$, $\log_3(n)$

Very large

$O(n)$ - Linear (n)

Double n: about 2x work.

Full scan; $3*n + 5$
Practice cue:
 $n = 200,000$

2 million / 200 million

$O(n \log n)$ - Linearithmic ($n \log n$)

Double n: slightly more than 2x work.

Merge sort
 $n*\log_2(n) + n$

~100,000 / ~8 million

$O(n^2)$ - Quadratic (n squared)

Double n: about 4x work.

All pairs
 $n*(n-1)/2$, $1+2+\dots+n$

~1,000 / ~14,000

$O(n^3)$ - Cubic (n cubed)

Double n: about 8x work.

Three full nested loops
 $n^3 + n^2*\log_2(n)$

~100 / ~500

$O(2^n)$ - Exponential (two to the n)

Add 1 to n: about 2x work.

All subsets (count)
 $2^n + n^2$

20 / 27

$O(n!)$ - Factorial (n factorial)

Add 1 to n: multiply work by n+1.

All permutations (count)
 $1*2*\dots*n$

9 / 11

Rounded size cues, not guarantees. Start with conservative estimates.